

Σ Equation Solver

A Java-based multi-type equation solving application

Introduction to OOP

Meri Papyan
Mari Amirjanyan
Anna Hovhannisyan

Linear · Quadratic · Cubic · Quartic ·
Exponential · Logarithmic
Rational · Absolute Value · Sine ·
Cosine · Tangent

Project Overview

What we built and how it's structured

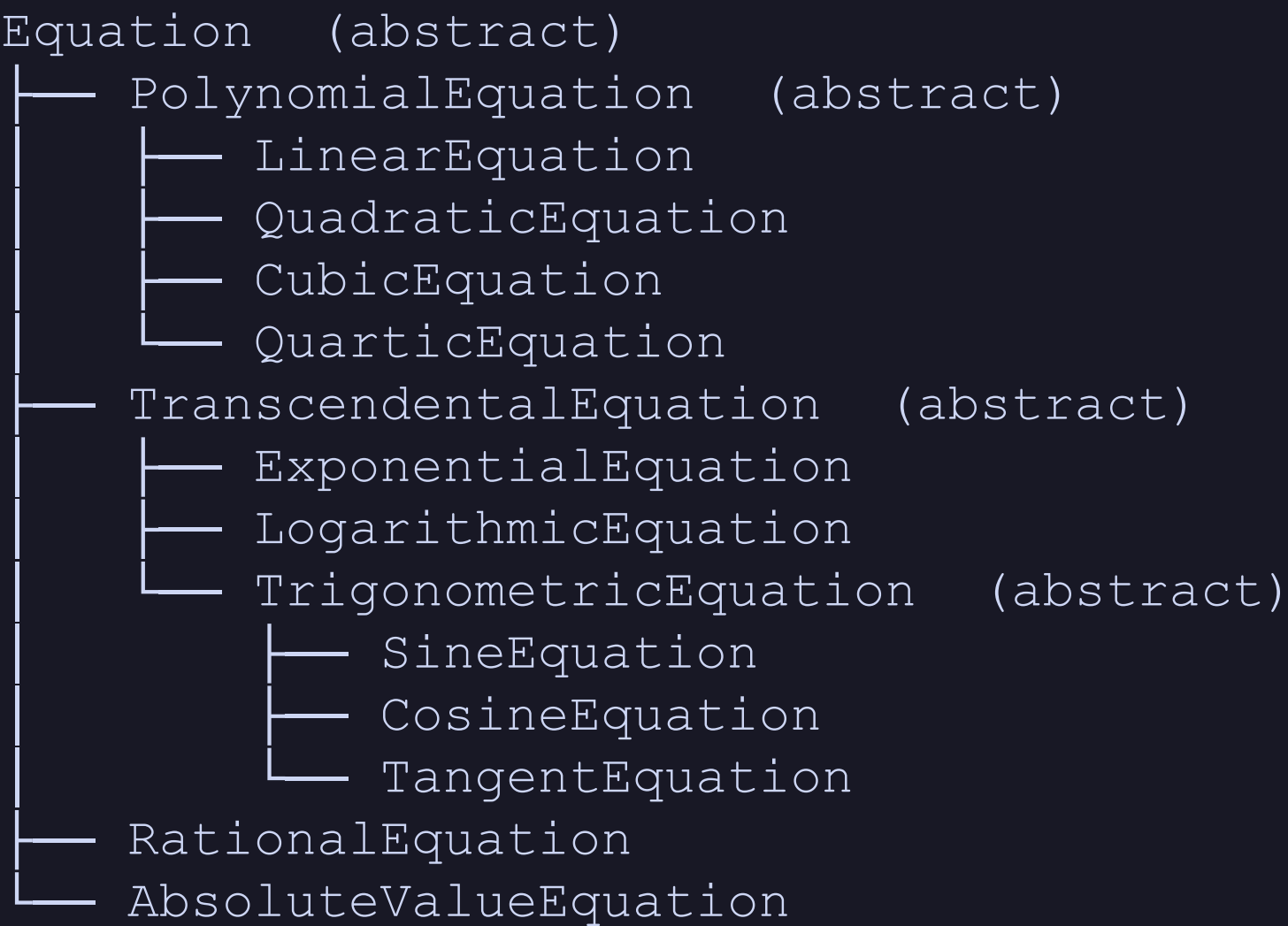
- Parses and solves 11 types of mathematical equations
- Full step-by-step solution output for every equation type
- Beautiful dark-themed Swing GUI
- Live equation type detection as you type
- History, copy-to-clipboard, ESC-to-clear interactions

Architecture: Model → Parser → Solver → Solution → Display

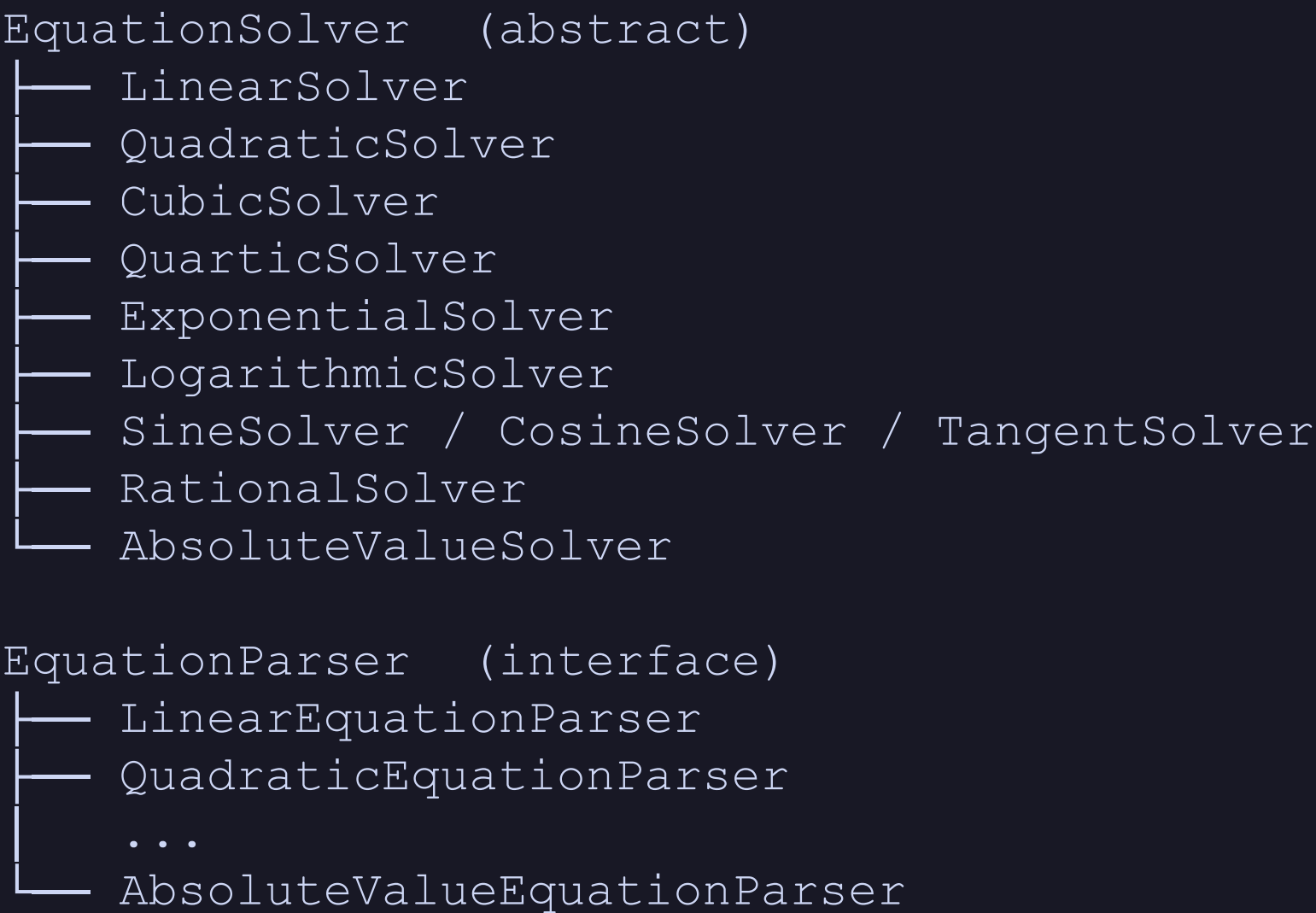
Class Hierarchy

Model layer — abstract classes and concrete types

Equation Model



Solver & Parser



Supported Equation Types

11 types fully implemented end-to-end

Linear

$$2x + 4 = 0$$

$$x = -2$$

Quadratic

$$x^2 - 5x + 6 = 0$$

$$x_1 = 3, \quad x_2 = 2$$

Cubic

$$x^3 - 6x^2 + 11x - 6 = 0$$

$$x_1=1, \quad x_2=2, \quad x_3=3$$

Quartic

$$x^4 - 5x^2 + 4 = 0$$

$$x_1=1, \quad x_2=-1, \quad x_3=2, \quad x_4=-2$$

Exponential

$$2^x = 8$$

$$x = 3$$

Logarithmic

$$\log_2(x) = 3$$

$$x = 8$$

Rational

$$(x^2-1) / (x-2) = 0$$

$$x_1=1, \quad x_2=-1$$

Absolute Value

$$|2x + 3| = 7$$

$$x_1=2, \quad x_2=-5$$

Sine

$$\sin(x) = 0.5$$

$$x_1=\pi/6, \quad x_2=5\pi/6$$

Cosine

$$\cos(x) = 0$$

$$x_1=\pi/2, \quad x_2=-\pi/2$$

Tangent

$$\tan(x) = 1$$

$$x=\pi/4$$

OOP Principles Applied

How each pillar appears in our codebase

Abstraction

`Equation` + `EquationSolver`

Abstract class `Equation` holds `rawInput` and `equationType` — concrete types add no fields, only behavior. `EquationSolver` declares abstract `doSolve()`; callers never touch the math directly.

Inheritance

`PolynomialEquation` → `LinearEquation`

Two abstract mid-layers (`PolynomialEquation`, `TranscendentalEquation`) share coefficient storage. `TrigonometricEquation` groups `Sine/Cosine/Tangent` under one shared solve strategy.

Polymorphism

`solver.solve(equation)`

`SolverFactory.getSolver()` returns `EquationSolver`. The GUI calls `.solve()` on that reference — it runs `QuadraticSolver` or `SineSolver` depending on type, with no if/else branching in the caller.

Encapsulation

`Solution` + `Parser` classes

`Solution` hides `roots[]`, `hasSolution`, and `steps[]` behind getters only — nothing writes fields directly. Each `Parser` owns its regex and coefficient logic; the factory sees only `parse(String) : Equation`.

Interfaces & Factory Pattern

EquationParser interface · ParserFactory · SolverFactory

EquationParser interface

```
interface EquationParser {  
    Equation parse(String input);  
}  
  
// 11 concrete implementations:  
// LinearEquationParser,  
// QuadraticEquationParser, ...
```

Each parser knows only its own equation format.
The interface guarantees a single contract: `parse()`.
New equation types extend the system without
changing any existing code (Open/Closed Principle).

Factory pattern flow

- 1 detectType(input)**
ParserFactory reads the raw string →
returns EquationType enum
- 2 getParser(type)**
Returns concrete parser,
e.g. QuadraticEquationParser
- 3 parser.parse(input)**
Returns typed Equation object
with coefficients extracted
- 4 solver.solve(equation)**
SolverFactory routes to correct
solver; returns Solution + steps

→ **Result:** GUI calls one method, gets back a fully solved Solution — zero knowledge of math internals required.

Design Pattern: Factory

ParserFactory and SolverFactory route by detected type

ParserFactory.java

```
public class ParserFactory {

    public static EquationType detectType(String input) {
        String s = input.replaceAll("\\s+", "").toLowerCase();
        if (s.startsWith("ln("))      return LOGARITHMIC;
        if (s.startsWith("sin("))     return SINE;
        if (s.contains("^x"))         return EXPONENTIAL;
        if (s.contains("|"))          return ABSOLUTE_VALUE;
        if (s.contains("/") && s.contains("(")) return RATIONAL;
        if (s.contains("x^4"))        return QUARTIC;
        if (s.contains("x^3"))        return CUBIC;
        if (s.contains("x^2"))        return QUADRATIC;
        return LINEAR;
    }

    public static EquationParser getParser(EquationType type) {
        return switch (type) {
            case LINEAR      -> new LinearEquationParser();
            case QUADRATIC   -> new QuadraticEquationParser();
            // ... all 11 types
        };
    }
}
```

Solve Flow

```
// Full flow in EquationApp / GUI:

EquationType type =
    ParserFactory.detectType(input);

EquationParser parser =
    ParserFactory.getParser(type);

Equation equation =
    parser.parse(input);

EquationSolver solver =
    SolverFactory.getSolver(type);

Solution solution =
    solver.solve(equation);

// Render steps + result
solution.displayWithSteps();
```

Step-by-Step Solutions

Every solver builds a human-readable trace

QuadraticSolver.java

```
// QuadraticSolver.doSolve()

List<String> steps = new ArrayList<>();
steps.add("Standard form:  $ax^2 + bx + c = 0$ , " +
        "a=" + a + ", b=" + b + ", c=" + c);
steps.add("Discriminant  $\Delta = b^2 - 4ac$ ");
double disc = b*b - 4*a*c;
steps.add(" $\Delta =$ " + disc);

if (disc < 0) {
    steps.add(" $\Delta < 0 \rightarrow$  no real roots");
    return Solution.noSolution(QUADRATIC, steps);
}

double x1 = (-b + Math.sqrt(disc)) / (2*a);
double x2 = (-b - Math.sqrt(disc)) / (2*a);
steps.add(" $x_1 =$ " + x1 + ",  $x_2 =$ " + x2);
return Solution.of(new double[]{x1,x2}, QUADRATIC, steps);
```

Output

$$x^2 - 5x + 6 = 0$$

Step 1 Standard form: $ax^2 + bx + c = 0$
 $a=1, b=-5, c=6$

Step 2 Discriminant $\Delta = b^2 - 4ac$

Step 3 $\Delta = (-5)^2 - 4 \times 1 \times 6 = 1$

Step 4 $\Delta > 0 \rightarrow$ two real roots

Step 5 $x = (-b \pm \sqrt{\Delta}) / 2a$

Step 6 $x_1 = 3.0, \quad x_2 = 2.0$

$$x_1 = 3, \quad x_2 = 2$$

GUI Features

Polished UX built on top of FlatLaf dark theme

●	Equation Types Panel	Click any type to load a working example instantly
●	Live Type Detection	Badge appears as you type — Quadratic, Rational, etc.
●	Step-by-Step Output	Numbered steps in styled HTML with colored result
●	History List	All solved equations listed, click to re-run
●	Copy Button	Copies full solution to clipboard, flashes Copied!
●	ESC to Clear	Clears input and hides badge instantly
●	Gradient Solve Button	Purple → cyan gradient, darkens on press

Thank You!

Questions?

11 equation types · Full GUI · Step-by-step solutions

Mari · Meri · Anna